



Steinerbäume

Modularbeit Graphentheorie

Julius Greppmair und Paul Menhart

25. September 2026

Inhaltsverzeichnis

Tabellenverzeichnis	iii
Abbildungsverzeichnis	iv
1. Einleitung	2
2. Methodik	3
3. Problemtaxonomie	4
4. Euklidisches Steinerbaum-Problem	5
4.1. Grundlagen	5
4.1.1. Vollständige und unvollständige Steinerbäume	5
4.1.2. Eigenschaften optimaler Lösungen	6
4.1.3. Anzahl der Topologien	6
4.2. Exakte Lösung	7
4.2.1. Algorithmus von Melzak	7
4.2.2. Implementierung	8
4.2.3. Komplexität	9
4.3. Näherung und Steiner-Verhältnis	9
4.3.1. MST und 120°-Heuristik	9
4.3.2. Steiner-Verhältnis	9
4.3.3. Aktuelle Entwicklungen	10
4.4. Experimente	10
4.4.1. Versuchsaufbau	10
4.4.2. Validierung	10
4.4.3. Laufzeit	10
4.4.4. Ursachen und Optimierungen	11
5. Rektilineares Steinerbaum-Problem	13
5.1. Hanan-Gitter und Reduktion auf das Graphenproblem	13
6. Steinerbaum-Problem in Graphen	14
6.1. NP-Vollständigkeit	14
6.2. Einfache Approximation über den minimalen Spannbaum	14
7. Implementierter Algorithmus	15
7.1. Laufzeitanalyse	15
7.1.1. Floyd-Warshall	16
7.1.2. Prim	16
7.1.3. Anzahl der aufgezählten Teilmengen	16
7.1.4. Pfadrekonstruktion	16
7.1.5. Metrische Hülle und der Zusammenhang der Teilmengen	17
7.2. Einfluss der Beschränkung auf $ S' \leq r - 2$	17

8. Nebenbemerkung: Dreyfus-Wagner	18
9. Eigenimplementierung und Vergleich mit Standardbibliothek	19
10. Experimenteller Aufbau	20
10.1. Technischer Aufbau	20
10.2. Instanzgenerator und Parameter	20
10.3. Ergebnisse	21
11. Fazit	26
Quellen	a

Tabellenverzeichnis

Tabelle 1	Anzahl vollständiger Topologien, aller Topologien (Steinerknoten mit Grad 3, Terminale mit Grad höchstens 3) und vollständiger Teilbäume über alle Teilmengen der Terminale.	7
-----------	---	---

Abbildungsverzeichnis

Abbildung 1	Laufzeit des exakten Lölers und der Heuristik (Median über die Seeds), logarithmische Achse.	11
Abbildung 2	Anteil realisierbarer Topologien und gültiger Seitenwahlen je Teilbaumgröße k	12
Abbildung 3	Gemessene Laufzeit des exakten Verfahrens gegenüber dem vorhergesagten Aufwand.	21
Abbildung 4	Laufzeit über der Knotenzahl bei sechs Terminalknoten, mit angepassten Exponenten.	22
Abbildung 5	Laufzeit über der Kantenwahrscheinlichkeit bei fester Graphgröße. . .	23
Abbildung 6	Laufzeit über der Anzahl der Terminalknoten bei 30 Knoten.	24
Abbildung 7	Verhältnis aus approximiertem und optimalem Gewicht, ein Punkt je Instanz, bei 30 Knoten. Für bessere Lesbarkeit sind die Punkte waagerecht leicht gestreut.	25

1. Einleitung

Gegeben ist eine Menge von Punkten, die miteinander verbunden werden sollen und zwar so günstig wie möglich. Verbindet man ausschließlich die gegebenen Punkte, so ist das der minimale Spannbaum, für den es seit Langem effiziente Verfahren gibt. Erlaubt man dagegen, zusätzliche Verzweigungspunkte einzuführen, die in der Aufgabenstellung gar nicht vorkommen, so wird die Verbindung in aller Regel kürzer, das Problem aber schwerer. Diese zusätzlichen Punkte heißen Steiner-Punkte und das Problem, die kürzeste Lösung zu finden das Steinerbaum-Problem [1].

Die Fragestellung tritt überall dort auf, wo ein Netz gebaut werden muss und Verzweigungen frei wählbar sind: bei der Verlegung von Leitungen, im Entwurf integrierter Schaltungen, in der Rekonstruktion von Stammbäumen in der Biologie. Entsprechend aktiv ist die Forschung bis heute [2].

Je nachdem, wo die zusätzlichen Punkte liegen dürfen und wie Abstände gemessen werden, entstehen verschiedene Varianten des Problems. Diese Arbeit behandelt drei davon in der Reihenfolge, in der sie auseinander hervorgehen: das euklidische Problem in der Ebene, das rektilineare Problem mit seiner Reduktion auf einen endlichen Graphen und schließlich das Steinerbaum-Problem in Graphen. Für die letzte Variante wurde ein exaktes Verfahren selbst implementiert und sein Laufzeitverhalten gemessen.

2. Methodik

Den Einstieg in das Thema bildete eine breite Recherche in allgemeinen Webquellen und in der Wikipedia. Ziel war dabei kein vollständiger Überblick, sondern eine erste Orientierung: Welche Problemstellungen werden unter dem Begriff des Steinerbaums zusammengefasst, worin unterscheiden sie sich und welche davon eignen sich für eine eigene Implementierung?

Als fachlicher Ausgangspunkt diente anschließend die Arbeit „Implications, conflicts, and reductions for Steiner trees“ von Rehfeldt und Koch [2]. Der Beitrag selbst behandelt Reduktionstechniken für exakte Löser und geht damit über den Umfang dieser Arbeit hinaus. Nützlich war vor allem seine Einleitung. Sie definiert das Steinerbaum-Problem in Graphen, ordnet den Stand der Technik ein und verweist auf relevante Wettbewerbe der letzten Jahre, über die sich aktuelle Fragestellungen erschließen lassen.

Näher betrachtet wurden zwei dieser Wettbewerbe, die 11. DIMACS Implementation Challenge von 2014 [3] und die PACE Challenge 2018 [4], [5]. Aus den Wettbewerbsunterlagen ergab sich als nächster Schritt eine Sichtung der verschiedenen Problemdefinitionen und ihrer Beziehungen zueinander. Entscheidend war dabei die Beobachtung, dass sich die geometrischen Varianten unter geeigneten Voraussetzungen auf das Problem in Graphen zurückführen lassen. Diese Erkenntnis bildete die Basis für die Wahl der behandelten Themen. Das euklidische und das rektilineare Problem werden theoretisch behandelt und über diese Reduktion mit dem Graphenproblem verbunden.

Auf dieser Grundlage wurde festgelegt, was selbst umgesetzt wird. Für das Problem in Graphen fiel die Entscheidung auf eine eigene Implementierung eines exakten Verfahrens, um dessen theoretisches Laufzeitverhalten nicht nur zitieren, sondern auch messen zu können. Als Vorlage dient Algorithmus 4.6.3 aus Jungnickel, „Graphs, Networks and Algorithms“ [6], der zugleich Grundlage der Lehrveranstaltung ist und das Verfahren einschließlich seiner Teilalgorithmen vollständig beschreibt.

3. Problemtaxonomie

(G, w)	Netzwerk aus einem zusammenhängenden Graphen $G = (V, E)$ und einer positiven Gewichtsfunktion w
$R \subseteq V$	Menge der Terminalknoten, die der Baum verbinden muss
$r = R $	Anzahl der Terminalknoten
$S = V \setminus R$	Steiner-Knoten: Knoten, die verwendet werden dürfen, aber nicht müssen
$S' \subseteq S$	die jeweils betrachtete Teilmenge der Steiner-Knoten
$w(T)$	Gewicht einer Kantenmenge T , also $\sum_{e \in T} w(e)$

4. Euklidisches Steinerbaum-Problem

Im euklidischen Fall des Steinerbaumproblems liegen die Terminale in der Ebene, und die Länge einer Kante entspricht dem euklidischen Abstand der verbundenen Knoten. Es gibt keine vorgegebene Menge möglicher Steinerknoten: Sie dürfen an beliebigen Stellen der Ebene platziert werden. Der Suchraum ist damit zunächst kontinuierlich und lässt sich nicht direkt durch Ausprobieren von Kandidaten durchsuchen.

Die Lösung wird daher in zwei Teilprobleme aufgeteilt. Zunächst wird die *Topologie* gewählt, also welche Knoten miteinander verbunden sind. Für eine Topologie lässt sich die optimale Lage der Steinerknoten anschließend exakt konstruieren, etwa mit dem Algorithmus von Melzak [7]. Die eigentliche Schwierigkeit liegt in der Wahl der Topologie: Ihre Anzahl wächst superexponentiell mit der Zahl der Terminale. Obwohl das Problem NP-schwer ist [8], lösen spezialisierte Verfahren wie GeoSteiner zufällige Instanzen mit über tausend Terminalen exakt, indem sie aussichtslose Teilbäume früh verwerfen [9].

Gleichzeitig ist der mögliche Vorteil gegenüber dem MST begrenzt. Nach der Vermutung von Gilbert und Pollak [1] ist ein minimaler Steinerbaum mindestens $\sqrt{3}/2 \approx 0,866$ -mal so lang wie der MST. Die Ersparnis beträgt demnach höchstens etwa 13,4 %. Auf zufälligen Instanzen liegt sie im Mittel sogar nur bei rund 3 % [9]. Es stellt sich die Frage wie nahe wir an die Optimale Lösung mit MST und einfachen Heuristiken kommen können und was bis welcher Terminalmenge wir noch die Optimale Lösung finden können.

4.1. Grundlagen

4.1.1. Vollständige und unvollständige Steinerbäume

Ein Steinerbaum ist *vollständig*, wenn alle Terminale Blätter sind (Grad 1). Hat mindestens ein Terminal Grad 2 oder 3, ist der Baum *unvollständig*. Damit ein unvollständiger Baum optimal sein kann, müssen alle Winkel an diesen Terminalen mindestens 120° betragen; sonst könnte der Baum durch einen zusätzlichen Steinerknoten verkürzt werden (siehe

Jeder unvollständige Baum lässt sich an seinen Terminalen mit Grad 2 oder 3 in vollständige Teilbäume zerlegen [10]. Für exakte Verfahren genügt es daher, vollständige Steinerbäume für Teilmengen der Terminale zu berechnen und diese anschließend zu kombinieren.

Beim Berechnen einer vollständigen Topologie kann ein Steinerknoten auf ein Terminal fallen, sodass die Kante zwischen beiden die Länge 0 hat. Der Baum ist dann *degeneriert*: Für diese Topologie existiert dann kein echter vollständiger Steinerbaum. Die

Kante der Länge 0 lässt sich kontrahieren, und es entsteht ein unvollständiger Baum gleicher Länge. Dieser ist der kürzeste Baum, der sich mit der ursprünglichen Topologie erreichen lässt, muss aber nicht Global optimal sein. Da er sich wiederum in vollständige Teilbäume kleinerer Teilmengen zerlegen lässt, die ohnehin betrachtet werden, können degenerierte Ergebnisse in exakten Verfahren verworfen werden.

4.1.2. Eigenschaften optimaler Lösungen

Ein minimaler Steinerbaum erfüllt die folgenden notwendigen Eigenschaften [1]:

1. **Winkel:** An jedem Knoten schließen je zwei Kanten einen Winkel von mindestens 120° ein.
2. **Grad:** Jeder Steinerknoten hat genau Grad 3; die Winkel zwischen seinen Kanten betragen also genau 120° . Terminale haben höchstens Grad 3.
3. **Anzahl:** Bei n Terminalen gibt es höchstens $n - 2$ Steinerknoten, in einem vollständigen Baum genau $n - 2$.

Begründung. (1) Schließen zwei Kanten va und vb einen Winkel unter 120° ein, ist v nicht der Fermat-Punkt des Dreiecks vab . Ersetzt man die beiden Kanten durch Verbindungen von v , a und b zum Fermat-Punkt, wird der Baum echt kürzer. (2) Ein Steinerknoten mit Grad 1 kann entfernt werden, einer mit Grad 2 lässt sich nach der Dreiecksungleichung durch eine gerade Kante ersetzen. Ab Grad 4 summieren sich mindestens vier Winkel zu 360° , sodass einer höchstens 90° beträgt, im Widerspruch zu (1). Dasselbe Argument begrenzt den Grad der Terminale. (3) Ein Baum mit n Terminalen und s Steinerknoten hat $n + s - 1$ Kanten. Da jede Kante zwei Endpunkte hat, gilt für die Gradsumme

$$\sum_{t \in R} \deg(t) + 3s = 2(n + s - 1), \quad \text{also} \quad s = n - 2 - \sum_{t \in R} (\deg(t) - 1).$$

Da jedes Terminal mindestens Grad 1 hat, folgt $s \leq n - 2$, mit Gleichheit genau dann, wenn alle Terminale Blätter sind.

Diese Eigenschaften gelten für den global optimalen Baum. Für den kürzesten Baum einer festen Topologie gilt nur die 120° -Regel an den (nicht degenerierten) Steinerknoten. Liegt dort an einem Terminal ein Winkel unter 120° , zeigt das, dass eine andere Topologie kürzer ist.

4.1.3. Anzahl der Topologien

Vollständige Topologien bauen wir schrittweise auf [10]. Eine vollständige Topologie mit $k - 1$ Terminalen hat $2k - 5$ Kanten. Um das k -te Terminal hinzuzufügen, wird eine beliebige Kante durch einen neuen Steinerknoten geteilt und das Terminal dort angehängt; dafür gibt es $2k - 5$ Möglichkeiten. Damit gibt es

$$\prod_{k=4}^n (2k-5) = 1 \cdot 3 \cdot 5 \cdot \dots \cdot (2n-5) = (2n-5)!!$$

vollständige Topologien (Doppelfakultät). Da jeder Schritt mit einem wachsenden Faktor multipliziert, wächst die Anzahl schneller als jede Exponentialfunktion c^n .

Lässt man auch unvollständige Topologien zu, steigt die Zahl nochmals deutlich (Tabelle 1, eigene Berechnung). Exakte Verfahren zählen diese jedoch nicht direkt auf: Da jeder unvollständige Baum in vollständige Teilbäume zerfällt, genügt es, vollständige Topologien für alle Teilmengen der Terminale zu betrachten und diese zu kombinieren. Die Zahl dieser Kandidaten $\sum_{k=2}^n \binom{n}{k} (2k-5)!!$ (mit einer einzelnen Kante für $k=2$) ist um Größenordnungen kleiner.

n	vollständig	alle Topologien	Teilbaum-Kandidaten
4	3	31	13
6	105	5.625	275
8	10.395	2.643.795	22.029
10	2.027.025	2.382.538.725	3.986.175

Tabelle 1: Anzahl vollständiger Topologien, aller Topologien (Steinerknoten mit Grad 3, Terminale mit Grad höchstens 3) und vollständiger Teilbäume über alle Teilmengen der Terminale.

4.2. Exakte Lösung

4.2.1. Algorithmus von Melzak

Melzak zeigte als Erster, dass sich das euklidische Steinerbaumproblem in endlich vielen Schritten exakt lösen lässt [7]. Sein Verfahren besteht aus zwei Teilen: einer geometrischen Konstruktion, die für eine feste vollständige Topologie den optimalen Baum liefert, und einer Suche über alle Topologien.

4.2.1.1. Konstruktion für eine feste Topologie

Vorwärts. Hat ein Steinerknoten s zwei Nachbarn a und b mit bekannter Position, wird über der Strecke ab ein gleichseitiges Dreieck errichtet. Seine dritte Ecke e ersetzt a , b und s und wird mit dem dritten Nachbarn c von s verbunden. Die Länge bleibt dabei erhalten: Für jeden Punkt s auf dem Kreisbogen über ab gegenüber von e gilt $|sa| + |sb| = |se|$. Die Ecke e muss auf der vom Rest des Baums abgewandten Seite von ab liegen. Da diese Seite ohne die Positionen der übrigen Steinerknoten nicht bekannt ist, werden beide Seiten getestet. Nach $k-2$ Schritten bleibt eine einzige Kante übrig, deren Länge der Länge des gesamten Baums entspricht.

Rückwärts. Die Schritte werden in umgekehrter Reihenfolge rückgängig gemacht. Der Steinerknoten s liegt jeweils im Schnittpunkt der Strecke ec mit dem Umkreis des

Dreiecks abe . Liegt dieser Schnittpunkt nicht auf der Strecke oder nicht auf dem Bogen zwischen a und b , existiert für diese Seitenwahl kein gültiger Baum. Scheitern alle Seitenwahlen, ist die Topologie nicht realisierbar und wird verworfen.

4.2.1.2. Vom vollständigen Teilbaum zum optimalen Baum

Da jeder Steinerbaum in vollständige Teilbäume zerfällt (<Abschnitt 4.1.1>), wird zunächst für jede Teilmenge X der Terminale der beste vollständige Teilbaum $\text{FST}(X)$ bestimmt. Der optimale Baum für X ist dann entweder dieser vollständige Teilbaum oder die Vereinigung zweier optimaler Bäume, die sich genau ein Terminal teilen:

$$\text{Best}(X) = \min \left\{ \text{FST}(X), \min_{t \in X, A \cup B = X, A \cap B = \{t\}} \text{Best}(A) + \text{Best}(B) \right\}.$$

Die Implementierung berechnet dies als dynamische Programmierung von kleinen zu großen Teilmengen, sodass jedes Ergebnis nur einmal berechnet wird.

4.2.1.3. Laufzeit

Für eine Topologie mit k Terminalen sind $k - 2$ Reduktionsschritte nötig, mit dem Testen beider Seiten also $O(k \cdot 2^{k-2})$ Operationen. Ist die zyklische Reihenfolge der Terminale im Baum bekannt, legt sie die Seiten eindeutig fest, und die Konstruktion gelingt in $O(k)$ [11]. Über alle Teilmengen ergeben sich

$$\sum_{k=2}^n \binom{n}{k} (2k - 5)!! \cdot 2^{k-2}$$

Konstruktionen. Die anschließende Kombination betrachtet $\sum_k \binom{n}{k} \cdot k \cdot 2^{k-1} = n \cdot 3^{n-1}$ Aufteilungen und ist damit vergleichsweise günstig. Die Laufzeit wird also von der superexponentiellen Zahl der Topologien dominiert.

4.2.2. Implementierung

Das Verfahren ist in Python mit NumPy und NetworkX umgesetzt [Link zum GitLab-Repository]. Die vollständigen Topologien werden wie in Abschnitt 4.1.3 erzeugt; für jede werden alle 2^{k-2} Seitenwahlen mit Melzak getestet und der kürzeste gültige Baum behalten. In der Rückwärtsphase wird ein Steinerknoten verworfen, wenn er nicht auf der Strecke ec liegt, der Winkel asb nicht 120° beträgt oder er mit einem Nachbarn zusammenfällt; degenerierte Bäume werden so gemäß Abschnitt 4.1.1 direkt aussortiert. Alle Vergleiche verwenden eine relative Toleranz von 10^{-9} , bezogen auf die beteiligten Kantenlängen.

Phase 1 berechnet den besten vollständigen Teilbaum für jede Teilmenge, Phase 2 kombiniert sie nach der Rekursion aus Abschnitt 4.2.1, von kleinen zu großen Teilmengen. Beide Phasen werden getrennt gemessen. Die 120° -Heuristik fügt jeweils beim ersten gefundenen Kantenpaar mit einem Winkel unter 120° einen Steinerknoten ein; den Fermat-Punkt bestimmt sie mit demselben Melzak-Verfahren für drei Terminale.

4.2.3. Komplexität

Das euklidische Steinerbaumproblem ist NP-schwer [8]. Aber nicht als NP-vollständig belegt. Es ist unbekannt ob die Entscheidungsvariante in NP liegt („Gibt es einen Steinerbaum der Länge höchstens L ?“), die Topologie und Koordinaten könnte man schnell konstruieren aber die Länge ist eine Summe von Quadratwurzeln. Ob eine solche Summe kleiner als L ist, lässt sich nicht bekanntermaßen in Polynomialzeit entscheiden, da unklar ist, wie viele Stellen dafür im Allgemeinen nötig sind (*Sum of Square Roots problem*).

Beispiel:

$$\sqrt{2} + \sqrt{34} + \sqrt{42} + \sqrt{55} + \sqrt{57} \quad \text{und} \quad \sqrt{5} + \sqrt{30} + \sqrt{37} + \sqrt{53} + \sqrt{58}$$

unterscheiden sich um nur etwa $1,43 \cdot 10^{-12}$.

GeoSteiner begegnet diesem Problem, indem es die Koordinaten der Ersatzpunkte exakt im Zahlkörper $\mathbb{Q}(\sqrt{3})$ berechnet und erst die Längen numerisch annähert [9]. In der Praxis ist diese Unterscheidung jedoch weitgehend irrelevant.

4.3. Näherung und Steiner-Verhältnis

4.3.1. MST und 120°-Heuristik

Der euklidische MST lässt sich in $O(n \log n)$ berechnen, da er in der Delaunay-Triangulierung der Terminale enthalten ist. Er ist bereits ein gültiger Steinerbaum ohne Steinerknoten. Eine einfache Heuristik verbessert ihn lokal: Schließen zwei Kanten an einem Knoten einen Winkel unter 120° ein, werden sie durch einen Steinerknoten im Fermat-Punkt der drei beteiligten Knoten ersetzt (vgl. Abschnitt 4.1.2). Das wird wiederholt, bis kein solcher Winkel mehr existiert. Da jeder Schritt den Baum verkürzt, ist das Ergebnis nie länger als der MST, es muss aber nicht optimal sein, da nur lokal verbessert wird und die Topologie des MST weitgehend erhalten bleibt. Wie nah die Heuristik dem Optimum kommt, untersucht TODO.

4.3.2. Steiner-Verhältnis

Wie viel ein Steinerbaum höchstens gegenüber dem MST spart, beschreibt das *Steiner-Verhältnis*. Gilbert und Pollak vermuteten $\rho = \sqrt{3}/2 \approx 0,866$ [1]; dieser Wert wird vom gleichseitigen Dreieck erreicht. Die Ersparnis betrüge dann höchstens etwa 13,4 %, und umgekehrt wäre der MST höchstens $2/\sqrt{3} \approx 1,155$ -mal so lang wie das Optimum.

Du und Hwang veröffentlichten einen Beweis der Vermutung [12], in dem später Lücken nachgewiesen wurden [13], [14]; die Vermutung gilt daher als offen. Bewiesen ist sie für $n = 4, 5, 6$ Terminale [15], [16], [17]. Die beste allgemein anerkannte untere Schranke

ist $\rho \geq 0,824$ [18]. Damit ist der MST bewiesenermaßen höchstens etwa 1,21-mal so lang wie der minimale Steinerbaum.

4.3.3. Aktuelle Entwicklungen

Ke et al. verwenden von großen Sprachmodellen erzeugte Beweisbausteine, die maschinell verifiziert werden, und behaupten damit eine untere Schranke von $\rho \geq 0,8559$ [19]. Dies würde die Schranke von 1985 deutlich verbessern und den MST-Faktor auf etwa 1,17 senken. [Status der Begutachtung prüfen.]

4.4. Experimente

4.4.1. Versuchsaufbau

Alle Messungen liefen auf [CPU, RAM] mit Python [Version], NumPy [Version] und NetworkX [Version]. Zufallsinstanzen bestehen aus n gleichverteilten Punkten im Einheitsquadrat mit den Seeds 0 bis 4 je Größe. Zur Validierung dienen die 46 Instanzen von Soukup und Chow aus der OR-Library (Datei `estein1`), deren Optima bekannt sind [20]. [Für größere n wird die Heuristik zusätzlich auf `estein10` bis `estein100` mit bekannten Optima getestet.]

4.4.2. Validierung

Der exakte Löser wurde auf allen 19 Instanzen aus `estein1` mit $n \leq 7$ ausgeführt. Die Abweichung vom bekannten Optimum betrug höchstens $3 \cdot 10^{-10}$, liegt also im Bereich der Rechengenauigkeit. Auch die berechneten MST-Längen stimmen mit den in der OR-Library hinterlegten Werten überein [max. Abweichung einsetzen]. Zusätzlich wurden zwei Plausibilitäten auf allen Läufen geprüft: Die Heuristik war nie kürzer als die exakte Lösung, und die Längenschränke veränderte das Ergebnis nie.

4.4.3. Laufzeit

Abbildung 1 zeigt die Laufzeit in Abhängigkeit von n . Für $n = 6$ benötigt der exakte Löser etwa 0,2 s, für $n = 7$ etwa 6,8 s und für $n = 8$ etwa 146 s. Die Zeit pro Melzak-Aufruf ist dabei nahezu konstant (etwa 0,15 ms für $n = 7$ und $n = 8$); die Laufzeit folgt also direkt der Zahl der Aufrufe aus Abschnitt 4.2.1. Daraus ergeben sich etwa 1 h für $n = 9$ und etwa 31 h für $n = 10$. Die exakte Lösung durch Aufzählung ist damit ab $n \approx 9$ unpraktikabel.

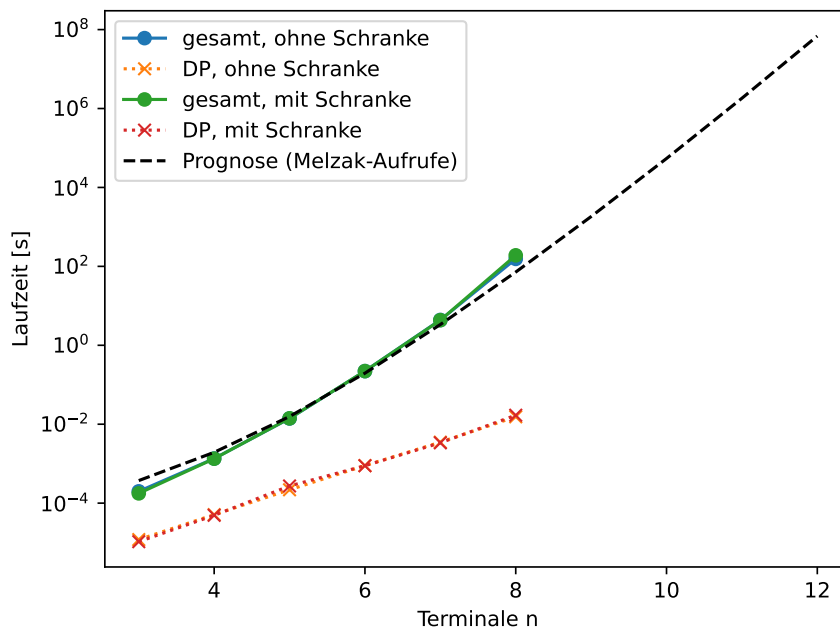


Abbildung 1: Laufzeit des exakten Löfers und der Heuristik (Median über die Seeds), logarithmische Achse.

4.4.4. Ursachen und Optimierungen

Die Kombination der Teilbäume (Phase 2) benötigt weniger als 1 % der Laufzeit; nahezu die gesamte Zeit entfällt auf die Melzak-Konstruktionen. Davon sind 99,9 % vergeblich, da für jede Topologie alle 2^{k-2} Seitenwahlen getestet werden, von denen höchstens eine gültig ist. Zudem ist nur ein kleiner und schnell sinkender Teil der Topologien überhaupt realisierbar (Abbildung 2): 19 % bei $k = 4$, 3,6 % bei $k = 5$, 0,9 % bei $k = 6$ und 0,09 % bei $k = 7$.

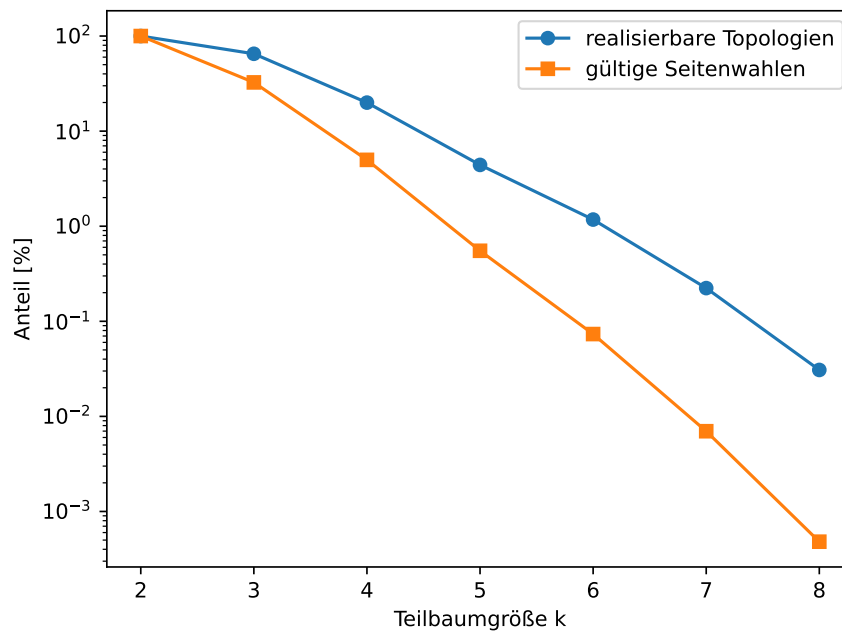


Abbildung 2: Anteil realisierbarer Topologien und gültiger Seitenwahlen je Teilbaumgröße k .

Als Optimierung wurde die Längenschranke aus [9] umgesetzt: Keine Kante eines minimalen Steinerbaums ist länger als die längste Kante des MST. Sie änderte die Laufzeit nicht messbar (Unterschiede zwischen -19% und $+9\%$ ohne Trend). Der Grund ist, dass die Kantenlängen erst in der Rückwärtsphase bekannt sind; die teure Vorwärtsphase läuft in jedem Fall vollständig. Wirksamer wären Maßnahmen, die ganze Topologien oder Seitenwahlen vorab ausschließen. Ist die zyklische Reihenfolge der Terminale bekannt, legt sie die Seiten eindeutig fest [11], was den Faktor 2^{k-2} beseitigt. GeoSteiner geht weiter und verwirft ganze Gruppen von Teilbäumen, meist ohne sie zu konstruieren [9]. Auf Implementierungsebene ließen sich zudem das Kopieren der Graphen pro Aufruf und das erneute Erzeugen der Topologien für jede Teilmenge gleicher Größe vermeiden; am exponentiellen Wachstum ändert das nichts.

liegt in der Größenordnung der rund 3% , die [9] für große Zufallsinstanzen angibt, und weit unter der theoretischen Obergrenze von $13,4\%$. Auf allen 46 Instanzen aus `estein1` ist die Heuristik im Mittel $0,8\%$ und höchstens $7,5\%$ länger als das Optimum. [Ergebnis für `estein10` bis `estein100` ergänzen: mittlere und maximale Abweichung, ggf. Trend mit n .]

5. Rektilineares Steinerbaum-Problem

Beim rektilinearen Problem dürfen Verbindungen nur waagerecht und senkrecht verlaufen. Diese Einschränkung wirkt zunächst willkürlich, ist aber die Geometrie, in der häufig Leiterbahnen auf Platinen und in integrierten Schaltungen verlegt werden.

5.1. Hanan-Gitter und Reduktion auf das Graphenproblem

Legt man durch jeden Terminalknoten je eine waagerechte und eine senkrechte Gerade, so entsteht ein Gitter, von dem gezeigt werden kann, dass ein optimaler rektilinear Steinerbaum darauf existiert. Diese Konstruktion geht auf Hanan zurück [21].

Damit wird aus einem Problem mit unendlich vielen Kandidatenpunkten eines mit endlich vielen. Bei r Terminalknoten hat das Gitter höchstens r^2 Schnittpunkte. Relevant ist dabei, dass sich diese Kandidatenpunkte allein aus der Lage der Terminalknoten ergeben und nicht davon abhängen, wie der Baum am Ende verläuft.

Die Konstruktion lässt sich noch verkleinern, da ein Gitterknoten, in dessen einem Quadranten kein Terminalknoten liegt, von keinem optimalen Baum als Verzweigung benötigt wird und daher entfernt werden kann.

Aus dem verbleibenden Gitter wird schließlich ein Graph. Die Schnittpunkte werden zu Knoten, die Gitterstrecken zwischen benachbarten Schnittpunkten zu Kanten und als Kantengewicht dient der Abstand der beiden Endpunkte. Die ursprünglichen Terminalknoten bleiben Terminalknoten. Ab diesem Punkt wird keine geometrische Information mehr benötigt und übrig bleibt genau das Steinerbaum-Problem in Graphen, um das es im Folgenden geht.

6. Steinerbaum-Problem in Graphen

Gegeben ist ein Netzwerk (G, w) mit zusammenhängendem Graphen $G = (V, E)$ und positiver Gewichtsfunktion w , dazu eine Menge $R \subseteq V$ von Terminalknoten. Gesucht ist ein Baum $T \subseteq E$ minimalen Gewichts $w(T)$, der alle Knoten aus R verbindet. Die übrigen Knoten $S = V \setminus R$ dürfen dabei verwendet werden, müssen es aber nicht.

Sind alle Knoten Terminalknoten, gilt also $R = V$, so ist der gesuchte Baum ein minimaler Spannbaum und mit den bekannten Verfahren in polynomieller Zeit zu bestimmen. Besteht R dagegen nur aus zwei Knoten, so ist die Lösung ein kürzester Weg. Schwierig wird erst der Bereich dazwischen, in dem auszuwählen ist, welche der Steiner-Knoten tatsächlich gebraucht werden.

6.1. NP-Vollständigkeit

Genau diese Auswahl macht das Problem schwer. Die Entscheidungsvariante, also die Frage, ob ein Baum mit Gewicht höchstens L existiert, gehört zu den ursprünglichen 21 Problemen, für die Karp die NP-Vollständigkeit gezeigt hat [22]. Das hier behandelte Optimierungsproblem, das nach dem tatsächlich minimalen Gewicht fragt, ist daher NP-schwer.

6.2. Einfache Approximation über den minimalen Spannbaum

Eine Näherungslösung kann in polynomieller Zeit gefunden werden, indem man die kürzesten Wege zwischen allen Terminalknoten sucht und auf diesen Abständen einen minimalen Spannbaum über R allein definiert. Anschließend ersetzt man jede seiner Kanten durch den zugehörigen kürzesten Weg. Das Ergebnis ist ein zulässiger Steinerbaum und sein Gewicht überschreitet das Optimum nie um mehr als den Faktor zwei.

In vielen Fällen ignoriert diese Konstruktion allerdings Steinerknoten, die zwar auf keinem direkten kürzesten Pfad zwischen zwei Terminalknoten liegen, aber effiziente Knotenpunkte für die Verbindung mehrerer Teilgraphen darstellen können.

7. Implementierter Algorithmus

Umgesetzt wurde Algorithmus 4.6.3 aus Jungnickel [6]. Er bestimmt den minimalen Steinerbaum exakt und beruht auf einer einfachen Überlegung: Stünde bereits fest, welche Steiner-Knoten im optimalen Baum vorkommen, so wäre nur noch ein minimaler Spannbaum über diese Knoten und die Terminalknoten zu bestimmen. Da das nicht feststeht, werden alle in Frage kommenden Teilmengen durchprobiert.

Der Ablauf besteht aus vier Schritten:

1. Berechne die kürzesten Wege zwischen allen Knotenpaaren.
2. Zähle alle Teilmengen $S' \subseteq S$ mit $|S'| \leq r - 2$ auf.
3. Bestimme für jede davon einen minimalen Spannbaum über $R \cup S'$, wobei als Kantengewicht der kürzeste Abstand zwischen zwei Knoten dient und merke dir den mit der niedrigsten Summe von Kantengewichten.
4. Ersetze die Kanten dieses Spannbaums durch die kürzesten Wege, für die sie stehen.

Der dritte Schritt arbeitet nicht auf dem ursprünglichen Graphen, sondern auf der *metrischen Hülle*: dem vollständigen Graphen über die ausgewählten Knoten, in dem das Gewicht einer Kante der kürzeste Abstand ihrer Endknoten im Originalgraphen ist. In dieser Hülle ist jede Kante vorhanden, weshalb jede Knotenteilmenge zusammenhängend ist und immer ein Spannbaum existiert.

```
max_steiner_count = min(len(terminals) - 2, len(steiner_vertices))
for i in range(0, max_steiner_count + 1):
    for selected_steiner_vertices in itertools.combinations(steiner_vertices, i):
        vertex_subset = terminals | set(selected_steiner_vertices)
        tree_edges, cost = prim(vertex_subset, distances)

        if cost < best_weight:
            best_weight = cost
            best_tree_edges = tree_edges
```

Listing 1: Kern des Verfahrens: Aufzählung der Teilmengen und Auswahl des minimalen Spannbaums.

7.1. Laufzeitanalyse

Insgesamt ergibt sich

$$O(|V|^3 + 2^{|S|} \cdot |R|^2).$$

Der erste Summand stammt aus der Berechnung der kürzesten Wege, der zweite aus der Aufzählung aller Teilmengen von Steinerknoten und der für jede davon nötigen Berechnung eines minimalen Spannbaums. Die folgenden Abschnitte begründen die einzelnen Anteile.

7.1.1. Floyd-Warshall

Die kürzesten Wege zwischen allen Knotenpaaren berechnet das Verfahren von Floyd und Warshall [23], [24]. Es besteht aus drei geschachtelten Schleifen über alle Knoten und kostet damit $O(|V|^3)$, unabhängig davon, wie viele Kanten der Graph hat. Es läuft genau einmal vor der Aufzählung, denn die Abstände ändern sich nicht mehr. Neben den Abständen liefert der Algorithmus auch die tatsächlich gewählten Pfade zwischen allen Knoten in Form einer Vorgängertabelle, was die Komplexität allerdings nicht erhöht.

7.1.2. Prim

Für die Spannbäume kommt eine Variante des Verfahrens von Prim [25] zum Einsatz. In jedem Schritt wird derjenige noch nicht aufgenommene Knoten gewählt, der dem bisherigen Baum am nächsten liegt und anschließend werden die Abstände der übrigen Knoten aktualisiert. Bei k Knoten sind das k Durchläufe mit je $O(k)$ Aufwand, insgesamt also $O(k^2)$. Innerhalb der Aufzählung wird dabei keine Wegsuche mehr ausgeführt. Die Abstände liegen bereits aus dem ersten Schritt vor und jeder Eintrag der Tabelle ist selbst ein kürzester Weg, sodass ein Umweg über weitere Knoten nie kürzer sein kann. Der ursprüngliche Graph wird also nicht mehr benötigt und Prim vergleicht nur noch Tabelleneinträge. Da die Tabelle einem vollständigen Graphen mit $\frac{k(k-1)}{2}$ Kanten entspricht, ist das einfache Absuchen mit $O(k^2)$ günstiger als eine Prioritätswarteschlange, die auf $O(k^2 \log k)$ käme.

Mit $k = |R \cup S'| \leq 2r - 2$ hängt dieser Aufwand allein von der Anzahl der Terminalknoten ab, nicht von der Größe des Graphen. Das erklärt den Faktor $|R|^2$ in der Gesamtkomplexität.

7.1.3. Anzahl der aufgezählten Teilmengen

Aufgezählt werden alle Teilmengen $S' \subseteq S$ mit $|S'| \leq r - 2$, also

$$\sum_{i=0}^{r-2} \binom{|S|}{i}$$

Stück. Nach oben ist das durch $2^{|S|}$ beschränkt. Entscheidend für das gemessene Verhalten ist, dass hier $|S| = n - r$ steht und nicht n . Je mehr Terminalknoten es gibt, desto weniger Steiner-Knoten bleiben übrig, über die aufgezählt werden könnte.

7.1.4. Pfadrekonstruktion

Die Kanten des gefundenen Spannbaums sind Kanten der metrischen Hülle. Sie stehen für kürzeste Wege im ursprünglichen Graphen und müssen zum Schluss wieder durch diese ersetzt werden. Dazu wird mit der Vorgängertabelle Kante für Kante rückwärts gelaufen, bis der Startknoten erreicht ist.

Da dieser Schritt nur ein einziges Mal läuft, nämlich nach der Aufzählung, fällt er für das Laufzeitverhalten nicht ins Gewicht. Der Baum besteht aus höchstens $2r - 3$ Kanten, jeder eingesetzte Weg ist höchstens $|V| - 1$ Kanten lang und jede davon kostet einen Zugriff auf die Tabelle, was $O(r \cdot |V|)$ Operationen ergibt.

7.1.5. Metrische Hülle und der Zusammenhang der Teilmengen

Im ursprünglichen Graphen muss eine beliebig gewählte Knotenteilmenge keinen zusammenhängenden Teilgraphen aufspannen. Der Vorteil der metrischen Hülle ist dagegen, dass jede Kante vorhanden ist und dadurch auch jede Teilmenge zusammenhängend ist. Damit entfällt eine Fallunterscheidung, die die Aufzählung der Teilmengen aufwändiger machen würde. Jede Teilmenge liefert einen Spannbaum und die Auswahl beschränkt sich darauf, den leichtesten davon zu speichern.

7.2. Einfluss der Beschränkung auf $|S'| \leq r - 2$

Die Aufzählung ist nicht über alle Teilmengen von S nötig, sondern nur über solche mit höchstens $r - 2$ Elementen [6], was den Aufwand bei wenigen Terminalknoten drastisch verkürzt. In der Praxis ist diese Ersparnis erheblich, ändert aber nichts an der Komplexitätsklasse, da der Aufwand mit wachsendem n und r weiterhin explodiert.

8. Nebenbemerkung: Dreyfus-Wagner

Das hier umgesetzte Verfahren ist nicht das einzige exakte. Bekannt ist auch der Ansatz von Dreyfus und Wagner [26], der stattdessen über Teilmengen der Terminalknoten iteriert und durch dynamische Programmierung Lösungskandidaten findet. Er läuft in $O(3^r \cdot n + 2^r \cdot n^2 + n^3)$ [27]. Es kann gezeigt werden, dass das Verfahren eine polynomielle Laufzeit hat, solange die Anzahl der Terminalknoten nur logarithmisch mit der Knotenzahl steigt.

Der Vergleich ist aufschlussreich, weil beide Verfahren exponentiell sind, aber in verschiedenen Größen. Dreyfus-Wagner wächst in der Anzahl der Terminalknoten r , Algorithmus 4.6.3 dagegen in der Anzahl der Steiner-Knoten $|S| = n - r$. Die beiden sind damit in genau entgegengesetzten Bereichen effizient.

Es liegt nahe, daraus ein Verfahren zu bauen, das zu Beginn beide Schranken vergleicht und jeweils das günstigere Vorgehen wählt. Der Aufwand beträgt dann

$$O(n^3 + \min(2^{n-r} \cdot r^2, 3^r \cdot n)).$$

Praktisch wäre das ein Gewinn, polynomiell wird es dadurch aber nicht. Dreyfus-Wagner ist nur polynomiell, solange r logarithmisch in n bleibt, die Aufzählung nur, solange $n - r$ logarithmisch bleibt. Wachsen die Anzahl an Terminalknoten r und die Anzahl an Steinerknoten $n - r$ gleichermaßen, bleibt auch das Minimum der beiden Schranken exponentiell.

Für Instanzen aus dem Hanan-Gitter ist dieses Verhältnis bereits festgelegt. Dort folgt die Knotenzahl aus der Zahl der Terminalknoten, es gilt $n \leq r^2$ und damit $|S| \leq r^2 - r$. Beide Parameter sind aneinander gekoppelt und solche Instanzen liegen unabhängig von der Eingabe stets im selben Bereich des Parameterraums. Selbst mit der Schranke $|S'| \leq r - 2$ bleibt $\binom{r^2 - r}{r - 2}$ die maßgebliche Größe, die schneller wächst als 3^r . Für das rektile Problem wäre Dreyfus-Wagner damit das passendere der beiden Verfahren.

9. Eigenimplementierung und Vergleich mit Standardbibliothek

In der Praxis kommen selten exakte Verfahren zum Einsatz, sondern Approximationen, oft in Verbindung mit Reduktionstechniken, die eine Instanz vor der eigentlichen Rechnung verkleinern [2]. Diese Verfahren werden hier nicht im Detail behandelt.

Als Vergleichswert in den Messungen dient eine Approximation aus der Python-Bibliothek `networkx` [28]. Sie liefert einen zulässigen Baum, dessen Gewicht das Optimum nie um mehr als den Faktor zwei übersteigt und dient als Bezugsgröße für Laufzeit und Güte.

10. Experimenteller Aufbau

10.1. Technischer Aufbau

Die Implementierung ist in Python geschrieben und kommt ohne Graphenbibliothek aus. Knoten sind ganze Zahlen, eine Kante ist ein Tripel aus ihren beiden Endknoten und ihrem Gewicht, normalisiert, so dass die kleinere Knotennummer zuerst aufgeführt wird. Ein Graph besteht aus einer Knotenmenge und einer Kantenmenge:

```
class Edge(_Edge):  
    u: int  
    v: int  
    weight: float  
  
class Graph:  
    vertices: set[int]  
    edges: set[Edge]
```

Listing 2: Die Datentypen, auf denen die Algorithmen arbeiten.

Darauf aufbauend sind die drei Bestandteile des Verfahrens einzeln implementiert: `floyd()` für die kürzesten Wege einschließlich Vorgängertabelle, `prim()` für die Spannbäume und `steiner()` für die Aufzählung samt Rückübersetzung der Wege. Die Vergleichsnäherung stammt aus `networkx`. Die eigenen Graphen werden dafür konvertiert.

Um den Aufbau herum liegen drei weitere Bausteine: ein Generator für Zufallsinstanzen, ein Script zur Messung der Laufzeiten und Abspeichern der Ergebnisse als CSV-Datei, sowie Code, der die Messdaten zu Grafiken weiterverarbeitet.

10.2. Instanzgenerator und Parameter

Die Messinstanzen sind zufällige zusammenhängende Graphen. Erzeugt werden sie in zwei Schritten: Zuerst entsteht ein zufälliger Spannbaum, indem jeder neue Knoten an einen bereits verbundenen angehängt wird. Anschließend wird jedes weitere Knotenpaar mit Wahrscheinlichkeit p zusätzlich verbunden. Der Umweg über den Spannbaum stellt den Zusammenhang sicher, ohne Graphen zu verwerfen und neu zu ziehen. Die Kantengewichte sind ganze Zahlen zwischen 1 und 10, die Terminalknoten werden zufällig aus den Knoten gezogen.

```
random.seed(seed)  
instance = random_steiner_graph(vertex_count, edge_probability, terminal_count)
```

Listing 3: Eine Instanz ist durch ihre Parameter und den Startwert vollständig bestimmt.

Damit hat jede Instanz drei Parameter: die Knotenzahl n , die Kantenwahrscheinlichkeit p und die Anzahl der Terminalknoten r . Der verwendete Startwert des Zufallsgenerators wird abgespeichert, sodass sich jede einzelne Messung aus ihrer Zeile in der Ergebnisdatei reproduzieren lässt.

Gemessen wird in drei Reihen, in denen jeweils ein Parameter variiert und die übrigen fest bleiben: über die Anzahl der Terminalknoten bei $n = 30$ und $p = 0,3$, über die Knotenzahl bei $r = 6$ und $p = 0,3$ sowie über die Kantenwahrscheinlichkeit bei $n = 20$ und $r = 5$. Jeder Parameterpunkt wird mit fünf verschiedenen Startwerten wiederholt, von denen jeweils der Median angegeben wird.

10.3. Ergebnisse

Alle Messungen verwenden zufällige zusammenhängende Graphen mit n Knoten, Kantenwahrscheinlichkeit p und r Terminalknoten. $|S| = n - r$ ergibt die übrigen Knoten.

Um abzuschätzen, ob die Implementierung plausibel ist, wird zunächst überprüft, ob die Laufzeit dem entspricht, was die theoretische Laufzeitkomplexität erwarten lässt. Abbildung 3 trägt die gemessene Laufzeit gegen den vorhergesagten Aufwand auf, also gegen n^3 für die kürzesten Wege zwischen allen Knotenpaaren zuzüglich eines Spannbaums je aufgezählter Teilmenge. Über 270 Läufe und vier Größenordnungen hinweg ergibt sich ein Exponent von 0,94 bei $R^2 = 0,99$. Die Messwerte folgen der Vorhersage also über den gesamten Bereich, wachsen aber etwas langsamer. Der Unterschied geht vermutlich auf einen konstanten Anteil zurück, der bei kleinen Instanzen überwiegt.

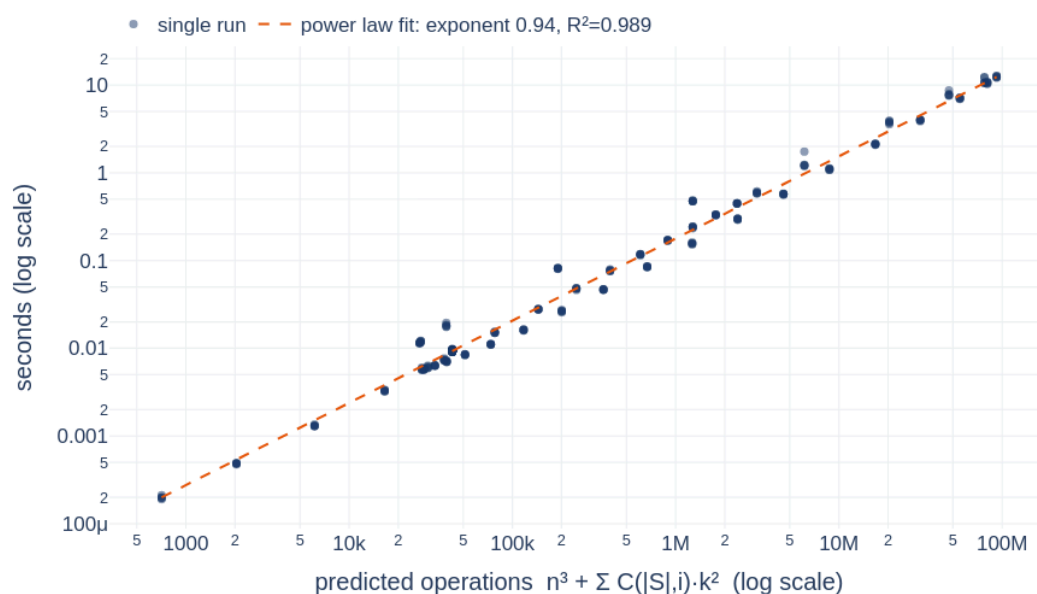


Abbildung 3: Gemessene Laufzeit des exakten Verfahrens gegenüber dem vorhergesagten Aufwand.

Wächst nur der Graph, während die Zahl der Terminalknoten fest bleibt, so bleibt die Laufzeit polynomiell, denn die Aufzählung ist durch $|S'| \leq r - 2$ beschränkt, sodass kein exponentieller Anteil entsteht. Abbildung 4 zeigt das für $r = 6$ mit einem gemessenen Exponenten von 5,5.

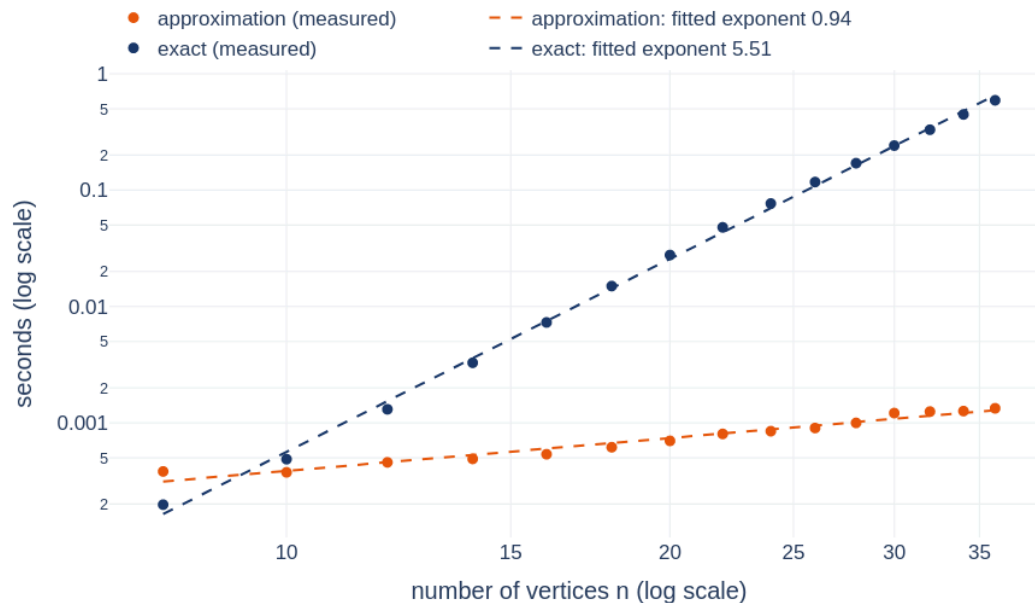


Abbildung 4: Laufzeit über der Knotenzahl bei sechs Terminalknoten, mit angepassten Exponenten.

Die Kantenwahrscheinlichkeit kommt in der Komplexität nicht vor, was zunächst unintuitiv wirkt. Abbildung 5 bestätigt es. Zwischen $p = 0,1$ und $p = 1$ bleibt das exakte Verfahren bei rund neun Millisekunden, während sich die Kantenanzahl verzehnfacht. Der Grund ist, dass nach dem ersten Schritt nicht mehr der Graph selbst, sondern die Distanzmatrix verarbeitet wird, was den Kanten in einem vollständigen Graphen entspricht. Die Approximation reagiert leicht auf die Dichte, weil sie auf den tatsächlichen Kanten arbeitet.

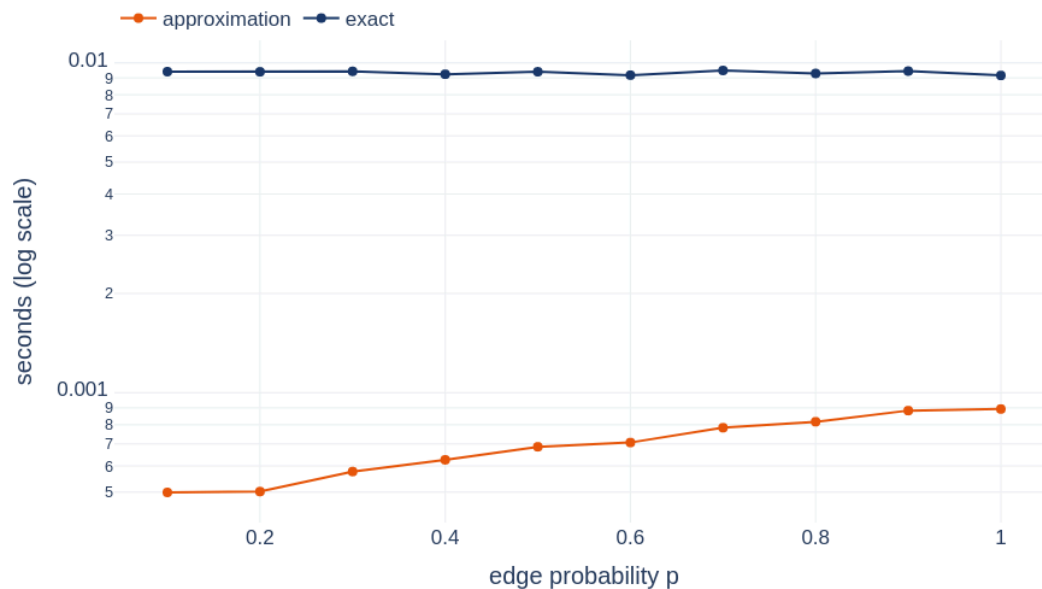


Abbildung 5: Laufzeit über der Kantenwahrscheinlichkeit bei fester Graphgröße.

Der eigentliche Kostentreiber ist die Anzahl der Terminalknoten. Abbildung 6 zeigt keinen monotonen Anstieg, sondern einen Buckel. Bei $r = 11$ steigt die Laufzeit des exakten Verfahrens bis auf etwa zwölf Sekunden an, fällt danach aber wieder auf sechs Millisekunden bei $r = 30$. Grund dafür ist, dass bei gleichbleibender Gesamtknotenzahl keine Steinerknoten mehr übrig bleiben wenn $r = 30$ erreicht ist. Das Problem reduziert sich dann auf das Finden des minimalen Spannbaums. Da der Suchalgorithmus Teilmengen von maximal $r - 2$ Steinerknoten absucht, ist auch bei niedrigen Werten von r die Laufzeit gering. Das Maximum liegt dort, wo das Produkt aus der Anzahl dieser Teilmengen und ihrer Größe am größten wird, also bei etwa einem Drittel der Knoten als Terminalknoten. Die Approximation bleibt über den gesamten Bereich im Millisekundenbereich.

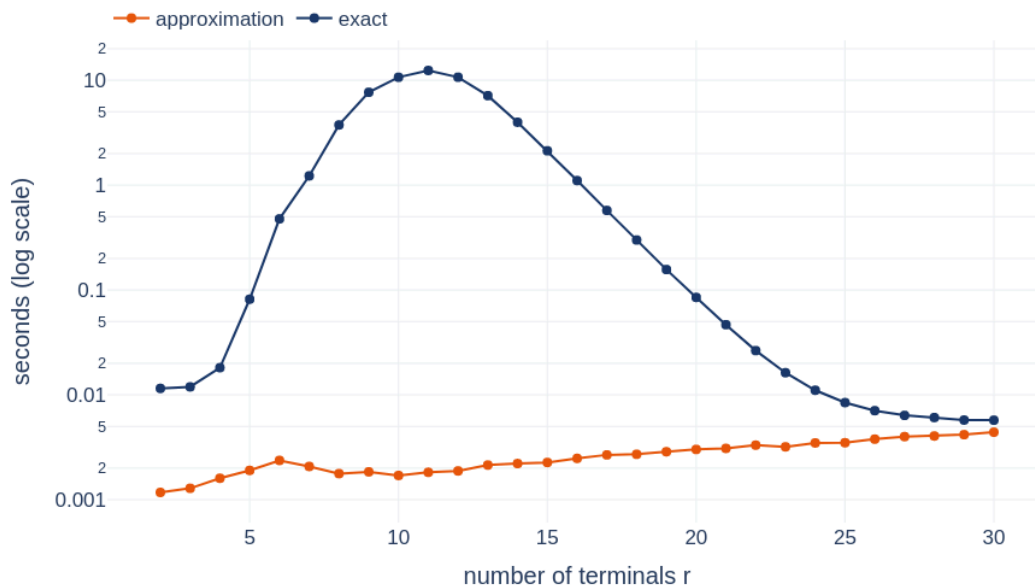
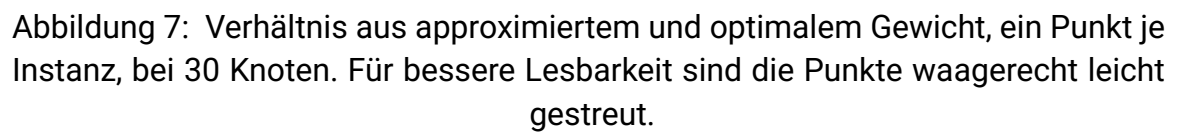


Abbildung 6: Laufzeit über der Anzahl der Terminalknoten bei 30 Knoten.

Zuletzt wurde gemessen, was die Approximation an Güte kostet. Abbildung 7 zeigt für dieselben Instanzen das Verhältnis aus approximiertem und optimalem Gewicht. In 59 Prozent der 145 Fälle findet die Approximation den optimalen Baum, im Mittel liegt sie 2,2 Prozent darüber, im schlechtesten Einzelfall 18 Prozent. Die garantierte Schranke von 2 wird damit nie erreicht.

Interessant ist, dass die schlechtesten Einzelfälle dort gruppieren, wo auch die Laufzeit des exakten Suchalgorithmus hoch ist. Die Erklärung ist dieselbe. Dort gibt es viele Steiner-Knoten, die tatsächlich etwas beitragen können und genau deren Auswahl trifft die Approximation nicht. Bei sehr wenigen Terminalknoten ist der gesuchte Baum nahezu ein kürzester Weg, bei sehr vielen nahezu ein minimaler Spannbaum. In beiden Fällen bleibt wenig zu entscheiden und die Näherung trifft das Optimum fast immer. Wie zu erwarten ist die Approximation also gerade dort am ungenauesten, wo das exakte Verfahren am teuersten ist.



11. Fazit

Betrachtet wurden drei Varianten des Steinerbaum-Problems. Im euklidischen Fall dürfen die Steinerknoten frei in der Ebene liegen, womit unendlich viele Lagen in Frage kommen und die Lösung über die Wahl einer Topologie und deren anschließende Konstruktion verläuft. Im rektilinearen Fall wird in der L_1 -Metrik gemessen, in der sich der Abstand zweier Punkte aus der Summe ihrer Koordinatenunterschiede ergibt. Das schränkt die möglichen Lagen so weit ein, dass ein endliches Gitter genügt, und genau über diese Reduktion geht die Variante in ein Graphenproblem über.

Alle drei Varianten sind NP-schwer [8], [22], [29]. Ein Verfahren, das für beliebige Eingaben in polynomieller Zeit eine optimale Lösung liefert, ist für keine von ihnen bekannt und eine garantiert optimale Lösung kostet damit exponentiellen Aufwand.

Die Messungen zeigen, wie ungleich sich dieser Aufwand über den Parameterbereich verteilt. Bei gleichbleibender Knotenzahl $n = 30$ steigt die Laufzeit des exakten Verfahrens mit wachsender Terminalzahl zunächst auf rund zwölf Sekunden bei $r = 11$ an und fällt danach wieder auf sechs Millisekunden bei $r = 30$. Der Grund liegt in der Aufzählung über Teilmengen der $n - r$ Steinerknoten, von denen bei vielen Terminalknoten nur wenige übrig bleiben.

Die approximative Lösung folgt diesem Verlauf nicht. Ihre Laufzeit bleibt über den gesamten Parameterbereich unter zehn Millisekunden. In 59 Prozent der Fälle findet sie den optimalen Baum, im Mittel weicht sie um 2,2 Prozent und im schlechtesten Einzelfall um 18 Prozent davon ab. Die größten Abweichungen treten bei mittleren Terminalzahlen auf, also dort, wo auch das exakte Verfahren am teuersten ist.

Für das Graphenproblem lässt sich daraus ablesen, in welchen Bereichen eine exakte Rechnung möglich bleibt und in welchen nur eine Näherung in Frage kommt. Das rektilineare Problem lässt sich über das Hanan-Gitter zwar auf einen Graphen zurückführen, dort sind Knoten- und Terminalzahl durch die Konstruktion aber fest aneinander gekoppelt. Solche Instanzen treffen immer denselben Ausschnitt des Parameterraums und die hier über frei gewählte r und n gemessenen Verläufe lassen sich nicht unmittelbar darauf übertragen. Für das euklidische Problem greift die Einschätzung noch weniger, da dort die Lage der Steinerknoten nicht im Voraus feststeht und die Anzahl der zu prüfenden Topologien immer exponentiell mit der Zahl der Terminalknoten wächst.

Quellen

- [1] E. N. Gilbert und H. O. Pollak, „Steiner Minimal Trees“, *SIAM Journal on Applied Mathematics*, Bd. 16, Nr. 1, S. 1–29, 1968, doi: 10.1137/0116001.
- [2] D. Rehfeldt und T. Koch, „Implications, conflicts, and reductions for Steiner trees“, *Mathematical Programming*, Bd. 197, S. 903–966, 2023, doi: 10.1007/s10107-021-01757-5.
- [3] DIMACS und ICERM, „11th DIMACS Implementation Challenge: Steiner Tree Problems“. [Online]. Verfügbar unter: <https://dimacs11.zib.de/>
- [4] É. Bonnet und F. Sikora, „PACE Challenge 2018“. [Online]. Verfügbar unter: <https://pacechallenge.org/2018/>
- [5] É. Bonnet und F. Sikora, „The PACE 2018 Parameterized Algorithms and Computational Experiments Challenge: The Third Iteration“, in *13th International Symposium on Parameterized and Exact Computation (IPEC 2018)*, C. Paul und M. Pilipczuk, Hrsg., in Leibniz International Proceedings in Informatics (LIPIcs), vol. 115. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2019, S. 26:1–26:15. doi: 10.4230/LIPIcs.IPEC.2018.26.
- [6] D. Jungnickel, *Graphs, Networks and Algorithms*, 1. Aufl., Bd. 5. in Algorithms and Computation in Mathematics, vol. 5. Berlin, Heidelberg: Springer, 1999. doi: 10.1007/978-3-662-03822-2.
- [7] Z. A. Melzak, „On the Problem of Steiner“, *Canadian Mathematical Bulletin*, Bd. 4, Nr. 2, S. 143–148, 1961.
- [8] M. R. Garey, R. L. Graham, und D. S. Johnson, „The Complexity of Computing Steiner Minimal Trees“, *SIAM Journal on Applied Mathematics*, Bd. 32, Nr. 4, S. 835–859, 1977, doi: 10.1137/0132072.
- [9] D. Juhl, D. M. Warme, P. Winter, und M. Zachariasen, „The GeoSteiner Software Package for Computing Steiner Trees in the Plane: An Updated Computational Study“, *Mathematical Programming Computation*, Bd. 10, Nr. 4, S. 487–532, 2018.
- [10] F. K. Hwang, D. S. Richards, und P. Winter, *The Steiner Tree Problem*, Bd. 53. in Annals of Discrete Mathematics, vol. 53. Amsterdam: Elsevier, 1992.
- [11] F. K. Hwang, „A linear time algorithm for full Steiner trees“, *Operations Research Letters*, Bd. 4, Nr. 5, S. 235–237, Feb. 1986, doi: 10.1016/0167-6377(86)90008-8.
- [12] D.-Z. Du und F. K. Hwang, „A Proof of the Gilbert–Pollak Conjecture on the Steiner Ratio“, *Algorithmica*, Bd. 7, Nr. 1–6, S. 121–135, 1992, doi: 10.1007/BF01758755.

-
- [13] N. Innami, B. H. Kim, Y. Mashiko, und K. Shiohama, „The Steiner Ratio Conjecture of Gilbert–Pollak May Still Be Open“, *Algorithmica*, Bd. 57, Nr. 4, S. 869–872, 2010, doi: 10.1007/s00453-008-9254-3.
- [14] A. O. Ivanov und A. A. Tuzhilin, „The Steiner Ratio Gilbert–Pollak Conjecture Is Still Open“, *Algorithmica*, Bd. 62, Nr. 1–2, S. 630–632, 2012, doi: 10.1007/s00453-011-9508-3.
- [15] H. O. Pollak, „Some Remarks on the Steiner Problem“, *Journal of Combinatorial Theory, Series A*, Bd. 24, Nr. 3, S. 278–295, 1978, doi: 10.1016/0097-3165(78)90058-4.
- [16] D.-Z. Du, F. K. Hwang, und E. Y. Yao, „The Steiner Ratio Conjecture Is True for Five Points“, *Journal of Combinatorial Theory, Series A*, Bd. 38, Nr. 2, S. 230–240, 1985, doi: 10.1016/0097-3165(85)90073-1.
- [17] J. H. Rubinstein und D. A. Thomas, „The Steiner Ratio Conjecture for Six Points“, *Journal of Combinatorial Theory, Series A*, Bd. 58, Nr. 1, S. 54–77, 1991, doi: 10.1016/0097-3165(91)90073-P.
- [18] F. R. K. Chung und R. L. Graham, „A New Bound for Euclidean Steiner Minimal Trees“, *Annals of the New York Academy of Sciences*, Bd. 440, Nr. 1, S. 328–346, 1985, doi: 10.1111/j.1749-6632.1985.tb14564.x.
- [19] Y. Ke, T. Huang, Y. Shu, D. He, J. Gai, und L. Wang, „Towards Solving the Gilbert–Pollak Conjecture via Large Language Models“. [Online]. Verfügbar unter: <https://arxiv.org/abs/2601.22365>
- [20] J. E. Beasley, „OR-Library: Distributing Test Problems by Electronic Mail“, *Journal of the Operational Research Society*, Bd. 41, Nr. 11, S. 1069–1072, 1990.
- [21] M. Hanan, „On Steiner's Problem with Rectilinear Distance“, *SIAM Journal on Applied Mathematics*, Bd. 14, Nr. 2, S. 255–265, 1966, doi: 10.1137/0114025.
- [22] R. M. Karp, „Reducibility Among Combinatorial Problems“, in *Complexity of Computer Computations*, R. E. Miller und J. W. Thatcher, Hrsg., Plenum Press, 1972, S. 85–103. doi: 10.1007/978-1-4684-2001-2_9.
- [23] R. W. Floyd, „Algorithm 97: Shortest Path“, *Communications of the ACM*, Bd. 5, Nr. 6, S. 345, 1962, doi: 10.1145/367766.368168.
- [24] S. Warshall, „A Theorem on Boolean Matrices“, *Journal of the ACM*, Bd. 9, Nr. 1, S. 11–12, 1962, doi: 10.1145/321105.321107.
- [25] R. C. Prim, „Shortest Connection Networks and Some Generalizations“, *The Bell System Technical Journal*, Bd. 36, Nr. 6, S. 1389–1401, 1957, doi: 10.1002/j.1538-7305.1957.tb01515.x.
- [26] S. E. Dreyfus und R. A. Wagner, „The Steiner Problem in Graphs“, *Networks*, Bd. 1, Nr. 3, S. 195–207, 1971, doi: 10.1002/net.3230010302.
-

- [27] M. Cygan u. a., *Parameterized Algorithms*. Springer, 2015. doi: 10.1007/978-3-319-21275-3.
- [28] A. A. Hagberg, D. A. Schult, und P. J. Swart, „Exploring Network Structure, Dynamics, and Function using NetworkX“, in *Proceedings of the 7th Python in Science Conference (SciPy 2008)*, G. Varoquaux, T. Vaught, und J. Millman, Hrsg., Pasadena, CA, USA, 2008, S. 11–15. doi: 10.25080/TCWV9851.
- [29] M. R. Garey und D. S. Johnson, „The Rectilinear Steiner Tree Problem is NP-Complete“, *SIAM Journal on Applied Mathematics*, Bd. 32, Nr. 4, S. 826–834, 1977, doi: 10.1137/0132071.